

Name: Adarsh Chandel

Concepts of Operating System

Assignment 2

Part A

What will the following commands do?

- `echo "Hello, World!"`

Prints Hello, World! to the terminal

- `name="Productive"`

assigns a string 'Productive' to the variable name

- `touch file.txt`

Creates a file named file.txt

- `ls -a`

Lists all the contents of the current directory, including all the hidden files, current directory and parent directory

- `rm file.txt`

Removes the file named file.txt

- `cp file1.txt file2.txt`

Copies the content of file1.txt to file2.txt, and creates the file2.txt, file, if it doesn't exist.

- `mv file.txt /path/to/directory/`

Moves file.txt to the specified directory

- `chmod 755 script.sh`

It gives read, write, execute permission to the user, read and execute to the groups and others.

- `grep "pattern" file.txt`

Prints all the lines in the file, file.txt containing the word pattern

- `kill PID`

Sends a signal to the process to terminate

- `mkdir mydir && cd mydir && touch file.txt && echo "Hello, World!" > file.txt && cat file.txt`

One step at a time. Makes directory mydir, then moves inside that directory, creates a file named file.txt, and writes to the file Hello, World! and then displays its content, in one step. && executes the next command only if previous only is true or better say here in this context, succeeds.

- `ls -l | grep ".txt"`

Lists all the files with the containing the string ".txt"

- `cat file1.txt file2.txt | sort | uniq`

Concatenates the two files, sorts them, and then prints all the unique lines

- `ls -l | grep "^d"`

Lists only the directories in the current directory. ^ indicates the start of line, and d is a literal, so it matches with all the files, starting with d.

- `grep -r "pattern" /path/to/directory/`

Searches recursively inside all the files and directories, to look for the word pattern, inside all the files, and once found, prints them with along with letting us which file has the pattern match found.

- `cat file1.txt file2.txt | sort | uniq -d`

Concatenates the two files, sorts them, and then prints all the duplicate lines that was common in both of them. Intersection, kinda thing.

- `chmod 644 file.txt`

Owner have read, write, groups and others have only read permissions. Nothing else.

- `cp -r source_directory destination_directory`

Copies a directory recursively with all its file and content inside

- `find /path/to/search -name "*.txt"`

In the specified path, searches for all the files ending with ".txt" or that is a txt file

- `chmod u+x file.txt`

- echo \$PATH

It prints the list of directories of the system environment path.

Part B

Identify True or False:

1. ls is used to list files and directories in a directory.

True

2. mv is used to move files and directories.

True, also used to rename

3. cd is used to copy files and directories.

False, it is the command for changing directory. Copy command is cp.

4. pwd stands for "print working directory" and displays the current directory.

True

5. grep is used to search for patterns in files.

True, say it pattern or some words in a line

6. chmod 755 file.txt gives read, write, and execute permissions to the owner, and read and execute permissions to group and others.

True, because when converted to binary it is of the form of 111-101-101 in respective to the file permission structure rwxrwxrwx

7. `mkdir -p directory1/directory2` creates nested directories, creating `directory2` inside `directory1` if `directory1` does not exist.

True, `-p` command, will ensure that the parent directory, `directory1` gets created if it doesn't exist, and then it will create `directory2`

8. `rm -rf file.txt` deletes a file forcefully without confirmation.

Technically valid, but `-r` is used only for deleting directories.

Identify the Incorrect Commands:

1. `chmodx` is used to change file permissions.

2. `cpy` is used to copy files and directories.

3. `mkfile` is used to create a new file.

4. `catx` is used to concatenate files.

5. `rn` is used to rename files.

All of them are incorrect. The correct command in order of the question would be `chmod`, `cp`, `touch` or `cat` command with re directional (`>`) operator, `mv` command

PART C

Question 1: Write a shell script that prints "Hello, World!" to the terminal.

```
cdac@NSL-LW-10001728: ~/Li × + v
#!/bin/bash
echo "Hello, World!"
~
~
~
```

```
cdac@NSL-LW-10001728:~/LinuxAssignments$ mkdir LabsAssignment
cdac@NSL-LW-10001728:~/LinuxAssignments$ mkdir Assignment2
cdac@NSL-LW-10001728:~/LinuxAssignments$ vi Q1.sh
cdac@NSL-LW-10001728:~/LinuxAssignments$ chmod +x Q1.sh
cdac@NSL-LW-10001728:~/LinuxAssignments$ ./Q1.sh
Hello, World!
cdac@NSL-LW-10001728:~/LinuxAssignments$ |
```

Question 2: Declare a variable named "name" and assign the value "CDAC Mumbai" to it. Print the value of the variable.

```
cdac@NSL-LW-10001728: ~/Li × + v
#!/bin/bash
name="CDAC Mumbai"
echo "$name"
~
~
```

```
cdac@NSL-LW-10001728:~/LinuxAssignments$ ls
Assignment2  Q1.sh  docs  duplicate.txt  file3.txt  input.txt  output.txt
LabsAssignment  data.txt  docs.zip  file1.txt  fruit.txt  numbers.txt
cdac@NSL-LW-10001728:~/LinuxAssignments$ cd Assignment2/
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ vi Q2.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ chmod +x Q2.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ ./Q2.sh
./Q2.sh: line 3: name: command not found

cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ vi Q2.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ ./Q2.sh
./Q2.sh: line 3: name: command not found

cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ vi Q2.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ ./Q2.sh
CDAC Mumbai
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ vi Q2.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ |
```

Question 3: Write a shell script that takes a number as input from the user and prints it.

```
cdac@NSL-LW-10001728: ~/Li × + v
#!/bin/bash

echo "User, please enter a number: "
read num
echo "You entered the number $num"
~
~

cdac@NSL-LW-10001728: ~/Li × + v
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ vi Q3.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ chmod +x Q3.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ ./Q3.sh
User, please enter a number:
32
You entered the number 32
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ |
```

Question 4: Write a shell script that performs addition of two numbers (e.g., 5 and 3) and prints the result.

```
cdac@NSL-LW-10001728: ~/Li × + v
#!/bin/bash

echo "Enter number 1: "
read num1

echo "Enter number 2: "
read num2

num3=$((num1+num2))
echo "Addition of numbers are $num3"
```

```
cdac@NSL-LW-10001728: ~/Li X + v
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ vi Q4.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ chmod +x Q4.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ ./Q4.sh
Enter number 1:
20
Enter number 2:
10
./Q4.sh: line 9: syntax error near unexpected token `('
./Q4.sh: line 9: `num3 = sum(( num1+num2 ))'
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ vi Q4.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ ./Q4.sh
Enter number 1:
20
Enter number 2:
10
./Q4.sh: line 9: syntax error near unexpected token `('
./Q4.sh: line 9: `num3=$sum((num1+num2))'
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ vi Q4.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ ./Q4.sh
Enter number 1:
20
Enter number 2:
10
Addition of numbers are 30
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ |
```

Question 5: Write a shell script that takes a number as input and prints "Even" if it is even, otherwise prints "Odd".

```
cdac@NSL-LW-10001728: ~/Li X + v
#!/bin/bash

echo -n "Please enter a number: "
read num

if [[ ($num%2 -eq 0) ]]; then
    echo "$num is even"
else
    echo "$num is odd"
fi
```

```
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ vi Q5.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ ./Q5.sh
Please enter a number: 89
89 is odd
```

Question 6: Write a shell script that uses a for loop to print numbers from 1 to 5.


```
cdac@NSL-LW-10001728: ~/Li × + ▾
#!/bin/bash

for i in {1..5}
do
    echo "$i"
done
~
~
```

```
cdac@NSL-LW-10001728: ~/Li × + ▾
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ vi Q6.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ chmod +x Q6.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ ./Q6.sh
1
2
3
4
5
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ vi Q6.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ |
```

Question 7: Write a shell script that uses a while loop to print numbers from 1 to 5.

```
cdac@NSL-LW-10001728: ~/Li × + ▾
#!/bin/bash

i=1
while [ $i -lt 6 ]
do
    echo "$i"
    ((i++))
done
~
~
```

```
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ vi Q7.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ ./Q7.sh
1
2
3
4
5
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ vi Q7.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ |
```

Question 8: Write a shell script that checks if a file named "file.txt" exists in the current directory. If it does, print "File exists", otherwise, print "File does not exist".

```
cdac@NSL-LW-10001728: ~/Li × + v
#!/bin/bash

echo -n "Enter the filename in the current directory: "
read filename

if [[ -e $filename ]]; then
    echo "$filename present in current directory"
else
    echo "$filename not present in current directory"
fi
~
~
```

```
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ vi Q10.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ vi Q8.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ ./ Q8.sh
-bash: ./: Is a directory
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ ./Q8.sh
Enter the filename in the current directory: Q10.sh
Q10.sh present in current directory
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ |
```

Question 9: Write a shell script that uses the if statement to check if a number is greater than 10 and prints a message accordingly.

```
cdac@NSL-LW-10001728: ~/Li × + v
#!/bin/bash

echo -n "Enter a number: "
read num

if [[ ($num -gt 10) ]]; then
    echo "$num is greater than 10"
else
    echo "$num is not greater than 10"
fi

~
```

```
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ vi Q9.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ chmod +x Q9.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ ./Q9.sh
Enter a number: 2
2 is not greater than 10
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ vi Q9.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ |
```

Question 10: Write a shell script that uses nested for loops to print a multiplication table for numbers from 1 to 5. The output should be formatted nicely, with each row representing a number and each column representing the multiplication result for that number.

```
cdac@NSL-LW-10001728: ~/Li × + v
#!/bin/bash

echo
echo
echo "Nested for loops to print multiplication table"
echo
for i in {1..5}
do
    echo "Multiplication table of $i is"

    for j in {1..10}
    do
        result=$(( i * j ))
        echo "$i * $j = $result"
    done
    echo " "
done
```

```
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ vi Q10.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ ./Q10.sh
```

Nested for loops to print multiplication table

Multiplication table of 1 is

```
1 * 1 = 1
1 * 2 = 2
1 * 3 = 3
1 * 4 = 4
1 * 5 = 5
1 * 6 = 6
1 * 7 = 7
1 * 8 = 8
1 * 9 = 9
1 * 10 = 10
```

Multiplication table of 2 is

```
2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
2 * 4 = 8
2 * 5 = 10
2 * 6 = 12
2 * 7 = 14
2 * 8 = 16
2 * 9 = 18
2 * 10 = 20
```

Multiplication table of 3 is

```
3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
3 * 10 = 30
```

Multiplication table of 4 is

```
4 * 1 = 4
4 * 2 = 8
4 * 3 = 12
4 * 4 = 16
4 * 5 = 20
4 * 6 = 24
4 * 7 = 28
4 * 8 = 32
4 * 9 = 36
4 * 10 = 40
```

Multiplication table of 5 is

```
5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45
5 * 10 = 50
```

```
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ vi Q10.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ |
```

Question 11: Write a shell script that uses a while loop to read numbers from the user until the user enters a negative number. For each positive number entered, print its square. Use the break statement to exit the loop when a negative number is entered.

```
cdac@NSL-LW-10001728: ~/Li X + v
#!/bin/bash

while true
do

    echo -n "Enter a number: "
    read num
    echo "$num is a positive number"
    square=$((num*num))
    echo "Square of entered number is $square"

    if [[ $num -le 0 ]]; then
        echo "$num is a negative number. Exiting....."
        break
    fi

done
```

```
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ vi Q11.sh
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ ./Q11.sh
Enter a number: 1
1 is a positive number
Square of entered number is 1
Enter a number: 2
2 is a positive number
Square of entered number is 4
Enter a number: 3
3 is a positive number
Square of entered number is 9
Enter a number: 5
5 is a positive number
Square of entered number is 25
Enter a number: 23
23 is a positive number
Square of entered number is 529
Enter a number: 12
12 is a positive number
Square of entered number is 144
Enter a number: -8
-8 is a positive number
Square of entered number is 64
-8 is a negative number. Exiting.....
cdac@NSL-LW-10001728:~/LinuxAssignments/Assignment2$ |
```

PART E

1. Consider the following processes with arrival times and burst times:

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	6

Calculate the average waiting time using First-Come, First-Served (FCFS) scheduling.

2. Consider the following processes with arrival times and burst times:

Process	Arrival Time	Burst Time
P1	0	3
P2	1	5
P3	2	1
P4	3	4

Calculate the average turnaround time using Shortest Job First (SJF) scheduling.

3. Consider the following processes with arrival times, burst times, and priorities (lower number indicates higher priority):

Process	Arrival Time	Burst Time	Priority
P1	0	6	3
P2	1	4	1
P3	2	7	4
P4	3	2	2

Calculate the average waiting time using Priority Scheduling.

4. Consider the following processes with arrival times and burst times, and the time quantum for Round Robin scheduling is 2 units:

Process	Arrival Time	Burst Time
P1	0	4
P2	1	5
P3	2	2
P4	3	3

Calculate the average turnaround time using Round Robin scheduling.

ANSWERS

Part C

① Calculate average WT using FCFS

Process	AT	BT
P ₁	0	5
P ₂	1	3
P ₃	2	6

Timeline: 0 5 8 14

Process execution order: P₁ (0-5), P₂ (5-8), P₃ (8-14)

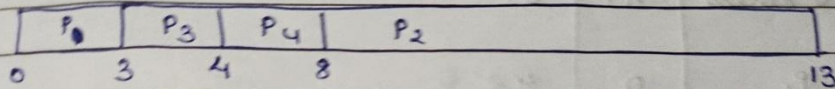
CT	TAT = CT - AT	WT = TAT - BT	RT
P ₁ 5	5 - 0 = 5	5 - 5 = 0	0 - 0 = 0
P ₂ 8	8 - 1 = 7	7 - 3 = 4	5 - 1 = 4
P ₃ 14	14 - 2 = 12	12 - 6 = 6	8 - 2 = 6

$$\overline{CT} = \frac{5+8+14}{3} = 9 \quad \overline{TAT} = \frac{24}{3} = 8$$

$$\overline{WT} = \overline{RT} = \frac{10}{3} = 3.33$$

2. Calculate average TAT using SJF:-

Process	AT	BT
P ₁	0	3
P ₂	1	5
P ₃	2	1
P ₄	3	4



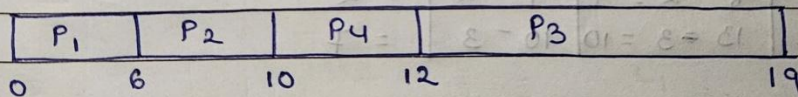
	CT	TAT	WT
P ₁	3	3 - 0 = 3	3 - 3 = 0
P ₂	13	13 - 1 = 12	12 - 5 = 7
P ₃	4	4 - 2 = 2	2 - 1 = 1
P ₄	8	8 - 3 = 5	5 - 4 = 1

$$WT = 9/4 =$$

$$\overline{CT} = 28/4 = 7 \quad \overline{TAT} = 22/4 = 5.5 \quad \underline{\underline{2.25}}$$

Q3. Processes

	AT	BT	Priority
P ₁	0	6	3
P ₂	1	4	1
P ₃	2	7	4
P ₄	3	2	2



	CT	TAT	WT
1	6	6 - 0 = 6	6 - 6 = 0
2	10	10 - 1 = 9	9 - 4 = 5
3	19	19 - 2 = 17	17 - 7 = 10
4	12	12 - 3 = 9	9 - 2 = 7

$$WT = 22/4 = 5.5$$

$$\overline{CT} = 47/4 = 11.75 \quad \overline{TAT} = 41/4 = 10.25$$

Q4. Process | AT | BT

P₁ 0 420

P₂ 1 531

P₃ 2 2

P₄ 3 31

$t_q = 2$ units

Fork ()

It is a child p

fork ()

Calculate the avg. TAT using Round Robin

$P_1 P_2 P_3 P_4 P_1 P_2 P_3 P_4 P_1 P_2$

P ₁	P ₂	P ₃	P ₁	P ₄	P ₂	P ₄	P ₂
0	2	4	6	8	10	12	14

CT	TAT	WT	WT
P ₁ 8	8 - 0 = 8	8 - 4 = 4	4
P ₂ 14	14 - 1 = 13	13 - 5 = 8	8
P ₃ 6	6 - 2 = 4	4 - 2 = 2	2
P ₄ 13	13 - 3 = 10	10 - 3 = 7	7

$$\overline{CT} = 41/4 = 10.25$$

$$\overline{TAT} = 8.75 \quad \overline{WT} = 21/4$$

$$= 5.25$$

(hello)